GRADE 10 · TERM 3

Loops, Strings & Number Detectives

A pencil-and-paper workbook: think it through first, learn the Delphi, then prove you can do it.

Name: _____

Class: _____ Date: _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Doing things again and again
2. Keep going until…
3. Strings from first principles
4. Number detectives
5. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 3 — FOR, WHILE and REPEAT loops; string handling from first principles; Ord and Chr; prime numbers, GCD and LCM.

UNIT 1 Doing things again and again

A computer is brilliant at repeating a task without getting bored. First you will see why that is useful, then you will meet the FOR loop.

By the end of this unit I can: (tick each box when you are sure)

- ☐ spot the part of a task that repeats
- ☐ explain what a loop is in my own words
- ☐ trace a FOR loop (counting up and down) by hand
- ☐ write a FOR loop in Delphi

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

1.1 The robot staircase

A robot understands only two commands: **UP** (one block up) and **RIGHT** (one block right). Write the commands it needs to climb a staircase with **5 steps** (each step is one UP and one RIGHT).
Then rewrite your instructions using the word **REPEAT** so that you write far less.

1.2 The tuck shop running total [4]

A pie costs R12. Complete the running total after each pie is bought.

Pies bought	Running total (R)
1	
2	
3	
4	
5	
6	
7	
8	

1.3 Spot the pattern [3]

Look at your table. **What is done again and again? What changes** each time? **What stays the same**?

LEARN IT

What is a loop?

A **loop** repeats a set of instructions. Before you write any loop, answer three questions:

1. What must be **repeated**?
2. How many times, or **until when**?
3. What **changes** each time round?

Delphi has three loops. The **FOR** loop is used when you know **how many times** to repeat.

LEARN IT

The FOR loop

iCount is the **counter**. It starts at 1 and Delphi adds 1 to it every time round, all by itself, until it passes 5. Everything between **begin** and **end** is the **loop body**. To count backwards write **downto** instead of **to**. You do not need begin...end if the body is only one statement.

Delphi

```
var
  iCount, iTotal : Integer;
begin
  iTotal := 0;
  for iCount := 1 to 5 do
  begin
    iTotal := iTotal + iCount;
  end;
  ShowMessage(IntToStr(iTotal));   // shows 15
end;
```

1.4 Fill in the blanks [4]

In a FOR loop the variable after the word **for** is called the _____. It increases by ____ every time round. We use a FOR loop when we know _____ to repeat. To count backwards we write _____ instead of **to**.

1.5 Be the computer: counting up [5]

Complete the table for the loop in *Learn It* (the FOR loop that adds 1 to 5). Write the value of **iTotal after** each pass.

Delphi

```
iTotal := 0;
for iCount := 1 to 5 do
  iTotal := iTotal + iCount;
```

iCount	iTotal after this pass
1	
2	
3	
4	
5	

1.6 Be the computer: counting down [4]

Trace this loop. Start with iResult = 100.

Delphi

```
iResult := 100;
for i := 4 downto 1 do
  iResult := iResult - i * 5;
```

i	iResult after this pass
4	
3	
2	
1	

1.7 Predict the output [3]

Write exactly what appears in **memOut** when this code runs.

Delphi

```
for i := 1 to 4 do
  memOut.Lines.Add(IntToStr(i) + ' x 3 = ' + IntToStr(i * 3));
```

1.8 **Write it: times table** **[4]**

Write a FOR loop that displays the **7 times table** from 1 to 10 in **memTable** (for example: *1 x 7 = 7*).

1.9 **Write it: Gauss's trick** **[4]**

Write the code to add up all the whole numbers from **1 to 100** and display the answer in **lblTotal**. Include your variable declarations. (The answer should be 5050.)

1.10 **Write it: countdown** **[3]**

Display the numbers **10 down to 1** in **memOut**, and then the words **Lift off!** once the loop has finished.

UNIT 2 Keep going until…

Sometimes nobody knows in advance how many times a task must be repeated. You keep going *until* something happens. This is where WHILE and REPEAT come in.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain the difference between WHILE and REPEAT
- ☐ choose the correct loop for a problem
- ☐ trace WHILE and REPEAT loops
- ☐ find mistakes that stop a loop from ending
- ☐ use a sentinel value to stop a loop

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

2.1 Guess my number

Play with a partner. Your partner secretly writes a number from **1 to 20** on a scrap of paper. You guess; they answer **HIGHER**, **LOWER** or **CORRECT**. Write each guess below. Play twice.

Round 1 guesses: _____ Round 2 guesses: _____

Could you have known in advance how many guesses you would need? What made you stop?

THINK FIRST · no computer needed

2.2 The ATM

Write, in everyday English, the steps an ATM follows to check your PIN when you might type it wrong. Use the words **keep** and **until**.

LEARN IT

WHILE and REPEAT

WHILE checks its condition **first**. It keeps going **while** the condition is True, so it may run **zero** times. **REPEAT...UNTIL** checks **last**. It keeps going **until** the condition becomes True, so it always runs **at least once**. A special value that tells the loop to stop (like 0 or -1) is called a **sentinel**. Inside the loop you must **change** something, otherwise the condition never changes and the loop never ends.

Delphi

```
// WHILE: test first. Sentinel = 0
iNum := StrToInt(InputBox('Input', 'Number (0 to stop):', ''));
while iNum <> 0 do
begin
  iTotal := iTotal + iNum;
  iNum := StrToInt(InputBox('Input', 'Number (0 to stop):', ''));
end;

// REPEAT: test last. Keeps asking until the age is valid
repeat
  iAge := StrToInt(InputBox('Age', 'Enter your age (14 to 19):', ''));
until (iAge >= 14) and (iAge <= 19);
```

2.3 Compare the loops [6]

Complete the table.

	WHILE ... DO	REPEAT ... UNTIL
When is the condition tested?		
Fewest times the body can run		
The loop continues while the condition is...		

2.4 Which loop? [6]

Circle the loop that suits each problem best.

1	Display the numbers 1 to 50.	**FOR** **WHILE** **REPEAT**
2	Keep asking for a password until it is correct (it must be asked at least once).	**FOR** **WHILE** **REPEAT**
3	Add up numbers until the user types -1. The user might type -1 straight away.	**FOR** **WHILE** **REPEAT**
4	Print the 12 times table.	**FOR** **WHILE** **REPEAT**
5	Roll a die until a 6 comes up.	**FOR** **WHILE** **REPEAT**
6	Read marks until the user types 0, where 0 may be the very first entry.	**FOR** **WHILE** **REPEAT**

2.5 Be the computer: WHILE [6]

Trace this code. For each pass, write whether the condition is True or False **before** the body runs, then the values **after** the body. The last row is the check that stops the loop.

Delphi

```
iNum := 100;
iCount := 0;
while iNum > 10 do
begin
  iNum := iNum div 2;
  Inc(iCount);
end;
```

Check	iNum > 10 ?	iNum	iCount
1			
2			
3			
4			
5			

2.6 Be the computer: REPEAT [5]

The user types **12**, then **7**, then **20**, then **0**.

Delphi

```
iTotal := 0;
repeat
  iNum := StrToInt(InputBox('Input', 'Number (0 to stop):', ''));
  iTotal := iTotal + iNum;
until iNum = 0;
```

Typed	iNum	iTotal	iNum = 0 ?
12			
7			
20			
0			

2.7 Think critically [3]

What is the final value of iTotal? Now imagine the sentinel had been **-1** instead of 0 and the same code was used. What would go wrong, and how could you fix it?

2.8 Debug: the endless loop [3]

This code should add 1 + 2 + 3 + 4 + 5, but the program freezes and then fails to display a result. Find **three** errors and write the corrections.

Delphi

```
var
  iNum, iTotal : Integer;
begin
  iNum := 1;
  while iNum <= 5 do
  begin
    iTotal := iTotal + iNum;
  end;
  lblTotal.Caption := iTotal;
end;
```

2.9 Debug: roll a six [2]

This code should keep rolling a die until a 6 is rolled and then count the rolls. Find **two** errors.

Delphi

```
repeat
  iPick := Random(6) + 1;
  Inc(iRolls);
until iPick <> 6;
```

2.10 Write it: class average **[7]**

Write code that uses a loop to read marks with an InputBox until the user types **-1**. Then display the average in **lblAvg**. The -1 must **not** be counted. Only calculate the average if at least one mark was entered.

UNIT 3 Strings from first principles

A string is just a row of characters, each with a position number. If you can walk along that row with a loop, you can count, search, reverse and change text without any built-in shortcuts.

By the end of this unit I can: (tick each box when you are sure)

- ☐ describe how a computer looks at a string one character at a time
- ☐ use a loop with s[i] and Length
- ☐ use Ord and Chr
- ☐ build a new string from an old one

One thing I want to ask my teacher: _____

3.1 Be the computer

Put your pencil on box 1. Move along one box at a time and count the letter **E**, exactly as a computer would. How many E's are there? Then write the steps you followed in order.

1	2	3	4	5	6	7	8
R	E	M	E	M	B	E	R

3.2 Backwards

Start at box 5 and read backwards to box 1, writing each letter as you go. What word do you get?

1	2	3	4	5
L	O	G	I	C

LEARN IT

Strings, positions and a loop

In Delphi the **first character is at position 1** (not 0). **Length(s)** tells you how many characters there are, and **s[i]** gives the character at position **i**. A FOR loop from 1 to Length(s) visits every character. You can **build a new string** by adding characters to it with **+**.

Delphi

```
sWord := 'LOGIC';
for i := 1 to Length(sWord) do
  memOut.Lines.Add(IntToStr(i) + ': ' + sWord[i]);
// 1: L   2: O   3: G   4: I   5: C
```

LEARN IT

Ord and Chr

Every character has a number (its ASCII code). **Ord(c)** gives the number for a character and **Chr(n)** gives the character for a number. The letters of the alphabet are in order, so Chr(Ord('A') + 1) is 'B'. A lowercase letter is 32 more than its uppercase letter.

Delphi

```
iCode := Ord('A');          // 65
cNext := Chr(iCode + 1);    // 'B'
cLower := Chr(Ord('A') + 32);   // 'a'
```

3.3 ASCII codes [5]

Use Ord('A') = 65 and Ord('a') = 97 to complete the table. Count along the alphabet.

Character	Ord() value
A	
C	
F	
Z	
d	
0	
5	

3.4 Quick fire [5]

Ord('D') = _____. Chr(70) = _____. To turn the lowercase 'g' into uppercase, subtract _____. Chr(Ord('A') + 25) = _____. Ord('9') - Ord('0') = _____.

3.5 Be the computer: counting a letter [6]

Trace this code. Write *Yes* or *No* to show whether the letter is an E, and write the value of iCount after each pass.

Delphi

```
sWord := 'REMEMBER';
iCount := 0;
for i := 1 to Length(sWord) do
  if sWord[i] = 'E' then
    Inc(iCount);
```

i	sWord[i]	Is it 'E'?	iCount
1	R		
2	E		
3	M		
4	E		
5	M		
6	B		
7	E		
8	R		

3.6 Predict the output [3]

What is displayed in **lblOut**? How many times does the loop body run?

Delphi

```
sWord := 'ROBOT';
sRev := '';
for i := Length(sWord) downto 1 do
  sRev := sRev + sWord[i];
lblOut.Caption := sRev;
```

3.7 Write it: count the spaces **[4]**

A sentence is stored in **sSentence**. Write the loop that counts how many **spaces** it contains and stores the answer in **iSpaces**.

3.8 Write it: remove the spaces **[5]**

Write code that builds a new string **sResult** containing the same characters as **sSentence** but **without any spaces**.

3.9 Write it: a secret code **[5]**

Each letter of the uppercase word in **sWord** is replaced by the next letter in the alphabet (A becomes B, B becomes C, and so on). Write code that puts the coded word in **sCoded**. You may assume the word has no letter Z.

3.10 Challenge: wrap around

What goes wrong in the secret code if the word contains a **Z**? Rewrite the line inside your loop so that Z becomes A.

UNIT 4 Number detectives

Factors, primes and common divisors are old ideas from maths. Here you will first investigate them with pencil and paper, then teach the computer to do the same.

By the end of this unit I can: (tick each box when you are sure)

- ☐ list the factors of a number
- ☐ explain what a prime number is
- ☐ test for a prime using a loop and MOD
- ☐ find the GCD using the remainder method
- ☐ find the LCM from the GCD

One thing I want to ask my teacher: _____

4.1 Hunt for the primes [4]

A **prime number** has exactly two factors: 1 and itself. Use this method (invented by Eratosthenes over 2000 years ago):

1. Cross out 1. It is not prime.
2. Circle 2. Cross out every other multiple of 2.
3. Circle the next number that is not crossed out (3). Cross out every other multiple of 3.
4. Keep going until every number is circled or crossed out.

List the primes you found below.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30

THINK FIRST · no computer needed

4.2 Factor pairs

Find all the **factors** of 24 by writing them as pairs that multiply to 24 (for example 1 × 24). Then answer: how many factors does the prime number 7 have? How many does the number 1 have?

THINK FIRST · no computer needed

4.3 GCD and LCM by hand [4]

List the factors of 12 and of 18. The biggest factor they share is the **GCD** (greatest common divisor). Then list the first six multiples of 12 and of 18. The smallest multiple they share is the **LCM** (lowest common multiple).

LEARN IT

Teaching the computer

x is a factor of n when **n MOD x = 0** (no remainder). To test for a prime, **count the factors** by trying every number from 1 to n. A number is prime if it is greater than 1 **and** has exactly 2 factors.

Delphi

```
iCount := 0;
for iDivisor := 1 to iNumber do
  if iNumber mod iDivisor = 0 then
    Inc(iCount);
bPrime := (iNumber > 1) and (iCount = 2);
```

LEARN IT

The remainder method for GCD

To find the GCD of two numbers, keep replacing the pair with (second number, remainder) until the remainder is 0. The last non-zero number is the GCD. Then **LCM = (first DIV GCD) × second**.

Delphi

```
iA := 48;
iB := 18;
while iB <> 0 do
begin
  iRemainder := iA mod iB;
  iA := iB;
  iB := iRemainder;
end;
// iA now holds the GCD
```

4.4 Be the computer: is 12 prime? [8]

iNumber = 12. Complete the table for every value of iDivisor.

Delphi

```
iCount := 0;
for iDivisor := 1 to 12 do
  if 12 mod iDivisor = 0 then
    Inc(iCount);
```

iDivisor	12 MOD iDivisor	Factor?	iCount
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

4.5 Conclusion [2]

Is 12 a prime number? Use your table to explain.

4.6 Be the computer: remainder method [6]

iA starts at 48 and iB starts at 18. Complete one row for each pass of the loop.

```delphi
while iB <> 0 do
begin
  iRemainder := iA mod iB;
  iA := iB;
  iB := iRemainder;
end;
```

Pass	iRemainder	iA after	iB after
1			
2			
3			

4.7 From GCD to LCM [2]

State the GCD of 48 and 18 from your table. Then calculate the LCM using **(48 DIV GCD) × 18**.

4.8 Write it: list the factors [5]

Write code that displays all the factors of **iNumber** in **lblOut**, separated by spaces (for 12 it would show *1 2 3 4 6 12*).

4.9 Challenge: all the primes up to 30

Use **nested loops** to display every prime number from 2 to 30 in **memOut**. Think: the inner loop is the prime test you already wrote. What must be reset for every new number?

TERM CHECKPOINT

Loops, strings and numbers

Time: 45 minutes **Total:** 44 marks **Mark:** _____ / 44

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Trace and predict [12]

A1 [6]

Trace this code. Show the value of iTotal **after** each pass.

```delphi
iTotal := 0;
for i := 1 to 6 do
  if i mod 2 = 0 then
    iTotal := iTotal + i;
```

i	i MOD 2 = 0 ?	iTotal
1		
2		
3		
4		
5		
6		

A2 [3]

What is the final value of iNum? How many times did the loop body run?

```delphi
iNum := 3;
while iNum < 40 do
  iNum := iNum * 2;
```

A3 **[3]**

What does sOut contain at the end? Describe in one sentence what the code does.

```Delphi
sIn := 'GRADE';
sOut := '';
for i := 1 to Length(sIn) do
  sOut := sIn[i] + sOut;
```

SECTION B — Concepts [8]

B1 **[4]**

Circle the best loop for each problem.

1	Display every number from 1 to 20.	**FOR**	**WHILE**	**REPEAT**
2	Ask for a password until it is correct (at least one attempt).	**FOR**	**WHILE**	**REPEAT**
3	Read numbers until -1 is typed, which might be typed first.	**FOR**	**WHILE**	**REPEAT**
4	Display the numbers 10 down to 1.	**FOR**	**WHILE**	**REPEAT**

B2 **[2]**

Explain, in your own words, **one** difference between a WHILE loop and a REPEAT loop.

B3 **[2]**

Given that Ord('A') = 65 and Ord('a') = 97: Ord('C') = _____ and Chr(100) = _____.

SECTION C — Find the errors [6]

C1 [3]

This code should add the numbers 1 to 10. Find and correct **three** errors.

Delphi

```
var
  i, iSum : Integer;
begin
  for i = 1 to 10 do
  begin
    iSum := iSum + i;
  end;
  lblSum.Caption := iSum;
end;
```

C2 [3]

This code should count the vowels in sWord but fails. Find the error and write the corrected line.

Delphi

```
iCount := 0;
for i := 0 to Length(sWord) do
  if sWord[i] in ['A','E','I','O','U'] then
    iCount := iCount + 1;
```

SECTION D — Write the code [18]

D1 [6]

Write code that displays the **sum of all the odd numbers from 1 to 49** in **lblOut**.

D2 [6]

A learner's name is typed in **edtName**. Write code that counts how many times the letter **A** (uppercase **or** lowercase) appears in the name and displays the answer in **lblCount**.

D3 **[6]**

Use a **REPEAT** loop to keep asking for a mark with an InputBox until the mark is from 0 to 100 inclusive. Then display **Mark accepted** using ShowMessage.

COMING UP NEXT TERM · warm-up

A filing cabinet for the computer

In Term 4 you will make your program **remember** things by saving them in a file and reading them back later. Before the Delphi, think about an ordinary school office.

The secretary must **(a)** put a new learner's details in a folder in the cabinet and **(b)** take them out again next week. Write the steps for each job in order. Think about opening, reading or writing, and what happens when you have finished.